ENGINEERING BUILD PLAN · UPDATED

RajyaRank Student Mobile App

A native Flutter app for iOS and Android, built on top of the existing RajyaRank API and web platform — full feature parity with the student web experience.

LAST UPDATED	PLATFORM	STATUS
19 August 2026	**Flutter (Android live · iOS pending)**	**M0–M3 shipped · M4 next**

1 Executive Summary

RajyaRank already has a full student experience on the web — courses, tests, previous-year papers, doubts, payments — served by a NestJS API that a mobile app can plug straight into. This is not a rebuild; it is a native port, backed by a backend that turns out to already be closer to mobile-ready than most.

The scope for the first release is **full parity with the web app**: everything a student can do on rajyarank.com today, plus push notifications. Native Google Sign-In has been dropped from scope — the web app doesn't offer it either, so there's nothing to match parity against. The one other thing genuinely out of scope for this release is offline lesson/PDF downloads — that doesn't exist on web either, and is real new product surface rather than a straight port.

PROGRESS SINCE THE LAST VERSION OF THIS DOCUMENT

Four of eight milestones are now live in real Android builds tested on a physical device: Foundation (native branding, splash screen), Auth & Onboarding, Core Learning (courses, lesson player, study plan, revision), and Tests (timed attempt runner, results, review). Each milestone has gone through at least one round of real-device feedback and fixes before moving on — see §6.

2 What Ships in the First Release

Account & Auth

Email/password login (live), phone OTP login and signup (built, UI-gated the same way web currently gates OTP), password reset, onboarding wizard, session management.

Learning

Dashboard, enrolled courses & curriculum, lesson player (video / PDF / YouTube embed) with progress tracking, bookmarks, study plan, revision hub.

Tests

Test catalogue, timed attempt runner with question palette and autosave, submission, results, and a detailed per-question review.

Payments

Course & plan pricing, Razorpay checkout, coupon codes, saved cards, order history and receipts.

Account & Institutions

Profile, study goals, institution join/leave via access code, password management.

Engagement

Previous Year Question papers, Doubts (ask + teacher replies), Current Affairs, search, wishlist, support tickets.

DROPPED FROM SCOPE

Native Google Sign-In — not offered on the web app today, so there's no parity requirement driving it. Offline lesson/PDF downloads, real-time push for doubt replies (web is poll-based too), a companion tutor/staff app, and page-level PDF progress tracking remain deferred to a later release, unchanged from the original plan.

3 Technical Approach

CONCERN	CHOICE	STATUS
State management	`flutter_riverpod`	In use
Navigation	`go_router`	In use
HTTP client	`dio` + interceptors (Bearer auth, 401 refresh-retry)	In use
Token storage	`flutter_secure_storage`	In use
Video	`video_player` / `chewie`	In use
PDF viewer	`pdfx`	In use
YouTube / embeds	`webview_flutter`	In use
Payments	`razorpay_flutter`	M4
Push	`firebase_messaging`	M6
Visual design	RajyaRank's existing navy / orange / teal design tokens	In use
CI / CD	GitHub Actions + Fastlane	Not started

Builds today are produced and tested manually against the live production API; automation is planned once the payments milestone (M4) is underway.

4 Backend Changes Required

All additive — nothing the web app does today changes.

#	CHANGE	STATUS	WHY IT'S NEEDED
1	Return session tokens in the login/signup response body, not only as browser cookies	Shipped	The web app keeps using cookies unchanged; the mobile app stores these tokens itself since it has no cookie jar.
2	Accept a refresh token in the request body as a fallback to the cookie	Shipped	Lets the mobile app silently renew its session, and staff/admin sessions were hardened at the same time so their tokens are never echoed into a JSON body.
3	Native Google Sign-In token exchange	Dropped	No longer required — the feature itself is out of scope (§1).
4	Push notification device registration + sender	Planned — M6	The web app only has browser push today; mobile needs the FCM/APNs equivalent, wired into the existing notification fan-out.

5 Roadmap

M0 — **Foundation** Shipped

Project scaffold, real branding & native splash screen, navigation shell, secure auth, first backend changes.

M1 — **Auth & Onboarding** Shipped

Email/password login, phone OTP (UI-gated), signup, password reset (rebuilt to match web's in-app code flow exactly), the onboarding wizard.

M2 — **Core Learning** Shipped

Dashboard, courses & curriculum, the lesson player across video/PDF/embedded formats with progress heartbeats and completion gating, study plan, revision hub.

M3 — **Tests** Shipped

Catalogue, the timed attempt runner with question palette and autosave, submission, results, and detailed per-question review.

M4 — **Payments & Account** Next up

Course/plan purchase via Razorpay, saved cards, order history, profile and institution management.

M5 — **Engagement** Planned

Previous Year Questions, Doubts, Current Affairs, search, wishlist, support.

M6 — **Notifications** Planned

In-app notification centre and push notifications, with deep links into content.

M7 — **Polish & Submission** Planned

Accessibility and localization QA, crash reporting, performance pass, store listings, TestFlight/Play internal testing, submission.

6 Current Status

AVAILABLE NOW

A real Android build, tested end-to-end against the live production API: sign-in, an onboarding wizard, a live dashboard, enrolled courses with a full curriculum browser, a lesson player that plays video, renders PDFs, and embeds YouTube lessons with progress tracking, a 7-day study plan (mark done/skip/reschedule, regenerate), a revision hub (weak topics, in-progress and saved lessons), and a complete test-taking flow — timed attempts, autosave, submission, and a detailed scored review. Distributed as an installable `.apk` for direct device testing, ahead of any app-store listing.

BUILT FROM REAL-DEVICE FEEDBACK, NOT JUST THE ORIGINAL PLAN

- Native app icon and splash screen generated from RajyaRank's real logo, replacing the placeholder used in the very first test build.
- Login screen redesigned for more native-feeling interaction (animated tab switch, haptic feedback) after early feedback that the first pass felt static.
- Reset-password flow rebuilt to match the web app's actual behaviour exactly — a 6-digit code entered in-app next to the new password, not an emailed link, which the first build had gotten wrong.
- Verification-code fields locked to numeric-only, 6-digit input everywhere a code is entered.
- Logout now asks for confirmation before signing out.
- Phone-OTP login hidden from the sign-in screen so the app matches what's actually live on web today (email/password only); the code remains in place to re-enable later.
- A validation bug in the study-plan "reschedule" action — the app was sending a bare date where the server expects a full timestamp, causing a 422 error — found via production logs and fixed.

IOS IS BLOCKED ON ONE EXTERNAL STEP

Building an installable iOS app requires Apple's own toolchain (Xcode on macOS), which only runs through cloud build infrastructure with real Apple Developer Program credentials attached. That enrollment ($99/year, with identity verification that can take several days) still hasn't started — it is the single external, non-engineering dependency blocking iOS entirely, including a first on-device test install.

7 Next Steps

1. Start M4 (Payments & Account) — Razorpay checkout, saved cards, order history, profile and institution management.
2. Then M5 (Engagement) and M6 (Notifications) in sequence, each ending in a testable Android build.
3. Start Apple Developer Program enrollment — it still hasn't begun, and it gates every iOS deliverable regardless of how far engineering gets on Android.
4. Wire up GitHub Actions + Fastlane once payments are stable, to replace today's manual build-and-sideload process with automatic Play/TestFlight builds.